



DevOps

Where to start

Goals



selecting which value streams to start with



understanding the work being done in our candidate value streams



designing our organization and architecture with Conway's Law in mind



enabling market-oriented outcomes through more effective collaboration between functions throughout the value stream



protecting and enabling our teams



Choosing a value stream for DevOps transformation

Greenfield vs. Brownfield Projects



Greenfield project – a new software project or initiative, likely in the early stages of planning or implementation, where we build our applications and infrastructure from scratch, with few constraints



Brownfield project - existing products or services that are already serving customers and have potentially been in operation for years or even decades.

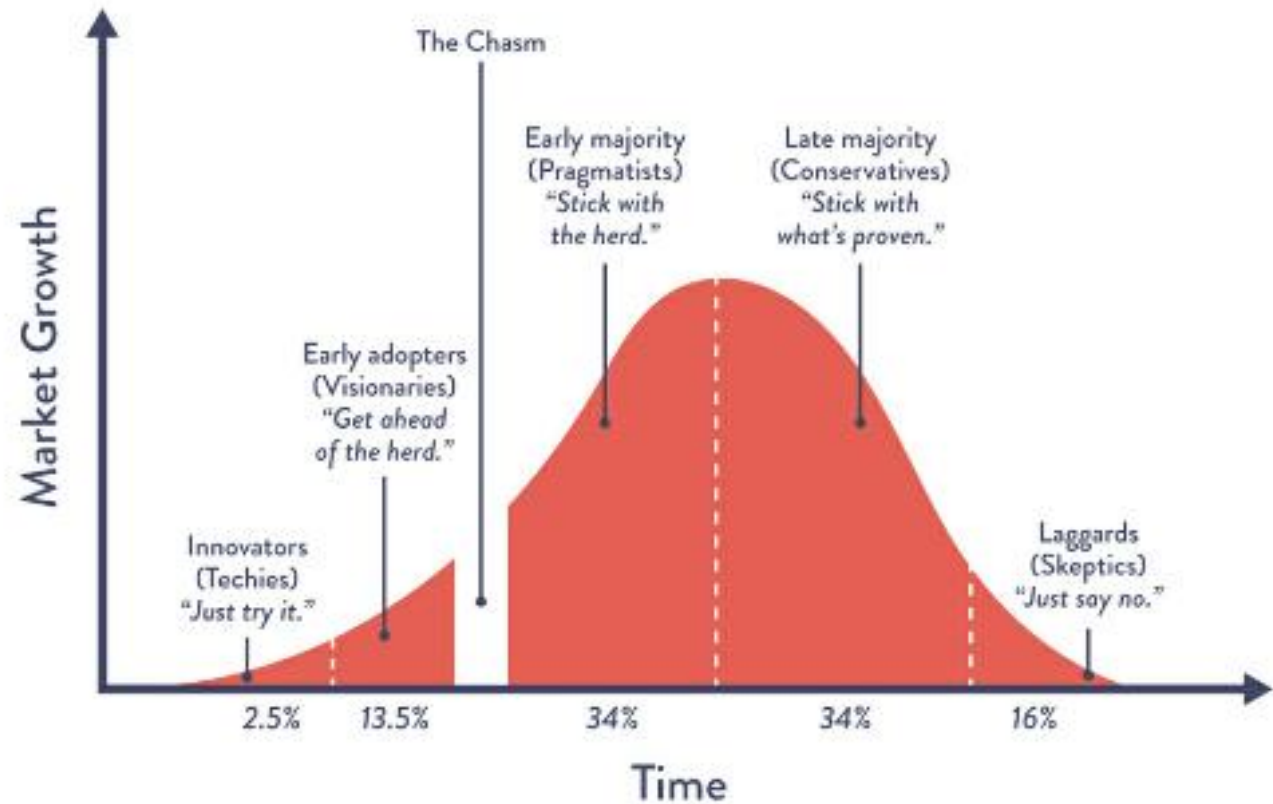
Brownfield impediments

running on unsupported platforms

no automated testing

tightly coupled architecture that prevents small teams from developing, testing, and deploying code independently

Start With the
Most
Sympathetic
and Innovative
Groups



Source: Moore and McKenna, Crossing the Chasm, 15

Expanding DevOps Across Our Organization



early wins and broadcast our successes



break up larger improvement
goals into small, incremental steps

fast improvement
early error detection

Ideal phases used by change agents to build and expand their coalition and base of support

find innovators and early adopters – teams who actually want to help, ideally people who are respected and have a high degree of influence over the rest of the organization => credibility

build critical mass and silent majority – expand DevOps practices to more teams and value streams with the goal of creating a stable base of support

Identify the holdouts - the high-profile, influential detractors who are most likely to resist (maybe even sabotage). Tackle this group only after achieving a silent majority and have established enough successes to successfully protect the initiative

Members of the value stream



Product owner: defines the next set of functionality



Development: develops application functionality



QA: ensures that feedback loops exist



IT Operations/SRE: maintains the production environment



Infosec: secures systems and data



Release managers: coordinates the production deployment and release processes



Technology executives or value stream manager: ensures that the value stream meets or exceeds the customer requirements from start to finish.

Create a Value Stream Map to See the Work



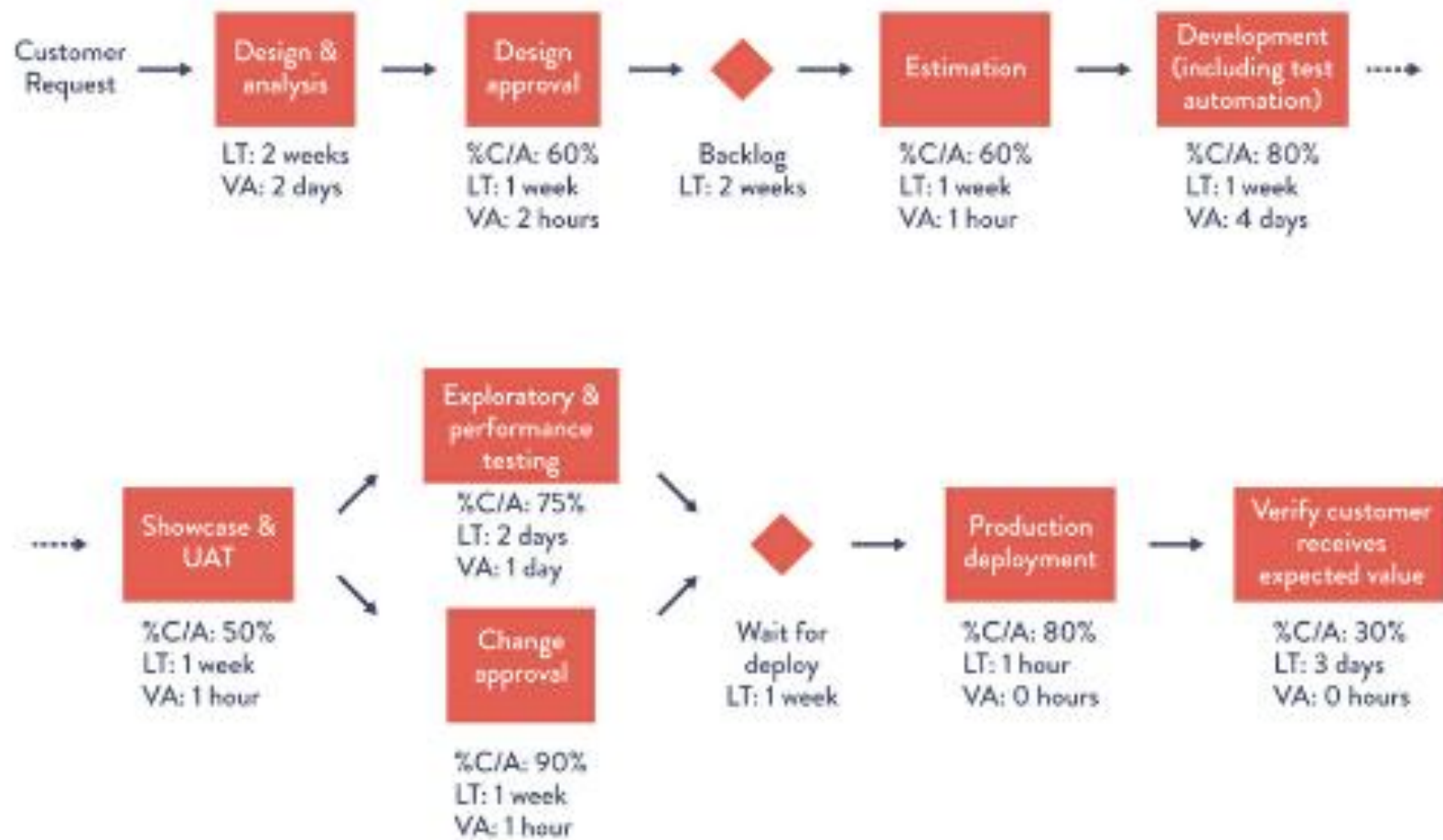
understand how work is performed



document as a value stream map



sufficiently understand the areas in the value stream that are jeopardizing the goals of fast flow, short lead times, and reliable customer outcomes



Aggregate values:

Total lead time: 10 weeks

Value added time: 7.5 days

Percent complete and accurate: 8.6%

Standard Software Development Value Stream

1

product owner:
customer request /
business
hypothesis

2

work is accepted
by Development

3

features are
implemented in
code and checked
into version control

4

builds are
integrated, tested
in a production-like
environment

5

builds deployed
into production =>
create value for
customers

Focus



places where work must wait weeks or even months



places where significant rework is generated or received

Metrics



%C/A – percentage complete accurate - reflects the quality of each process's output



VA – value added time - the time spent on activities that directly contribute to the creation or enhancement of a product or service for which a customer is willing to pay



LT – lead time - the amount of time from the start of a process until its conclusion

Creating a Dedicated Transformation Team



DevOps transformations are inevitably in conflict with ongoing business operations



#1 force: Bureaucracies that enforce approval processes, and controls to protect against variance



dedicated transformation team that is able to operate outside of the rest of the organization

How to create the team

Members to be solely allocated to the DevOps transformation efforts

Select team members who have skills across a wide variety of domains

Select team members who have longstanding and mutually respectful relationships with key areas of the organization

Create a separate physical space or virtual space, to maximize communication flow within the team and to create some isolation from the rest of the organization

Agree On a Shared Goal

measurable goal

clearly defined deadline (6 months - 2 years)

require considerable effort but still be achievable

achievement of the goal should create obvious value for the organization and for customers.

Goal examples



Reduce the percentage of the budget spent on product support and unplanned work by 50%



Ensure lead time from code check-in to production release is one week or less for 95% of changes



Ensure releases can always be performed during normal business hours with zero downtime



Integrate all the required information security controls into the deployment pipeline to pass all required compliance requirements

Keep Improvement Planning Horizons Short

flexibility and the ability to reprioritize and replan quickly

decrease the delay between work expended and improvement realized

faster integration of learnings into the next iteration

reduction in activation energy to get improvements

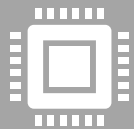
quicker realization of improvements that make meaningful differences

less risk that project is killed before can generate any demonstrable outcomes

Reserve 20% of Capacity for Non-Functional Requirements and Reducing Technical Debt + • ○



Organizations that don't pay down technical debt can find themselves so burdened with daily workarounds for problems left unfixed that they can no longer complete any new work

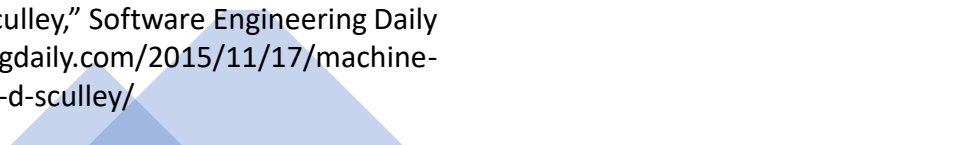


20% for non-functional requirements: maintainability, manageability, scalability, reliability, testability, deployability, and security



	User-visible	User-invisible
Positive Value	Feature	Architecture, non-functional requirements, process improvement
Negative Value	Defect	Technical debt

Source: "Machine Learning and Technical Debt with D. Sculley," Software Engineering Daily podcast, November 17, 2015, <http://softwareengineeringdaily.com/2015/11/17/machine-learning-and-technical-debt-with-d-sculley/>



Increase the Visibility of Work

the information displayed is up to date

constantly revise what is being measured to make sure it's helping to understand progress toward current target conditions

have a common backlog of work, stored in a common work system and using a shared vocabulary, so that work can be prioritized globally

rapid communication environment, e.g., chat rooms



Conway's Law

“organizations which design systems . . . are constrained to produce designs which are copies of the communication structures of these organizations. . . . The larger an organization is, the less flexibility it has and the more pronounced the phenomenon”



Implications of Conway's Law

how we organize our teams has a powerful effect on the software we produce, as well as our resulting architectural and production outcomes

Organizational Archetypes



Functional-oriented organizations optimize for expertise, division of labor, or reducing cost. They often have tall hierarchical organizational structures



Matrix-oriented organizations attempt to combine functional and market orientation. They often result in complicated organizational structures, such as individual contributors reporting to two managers or more



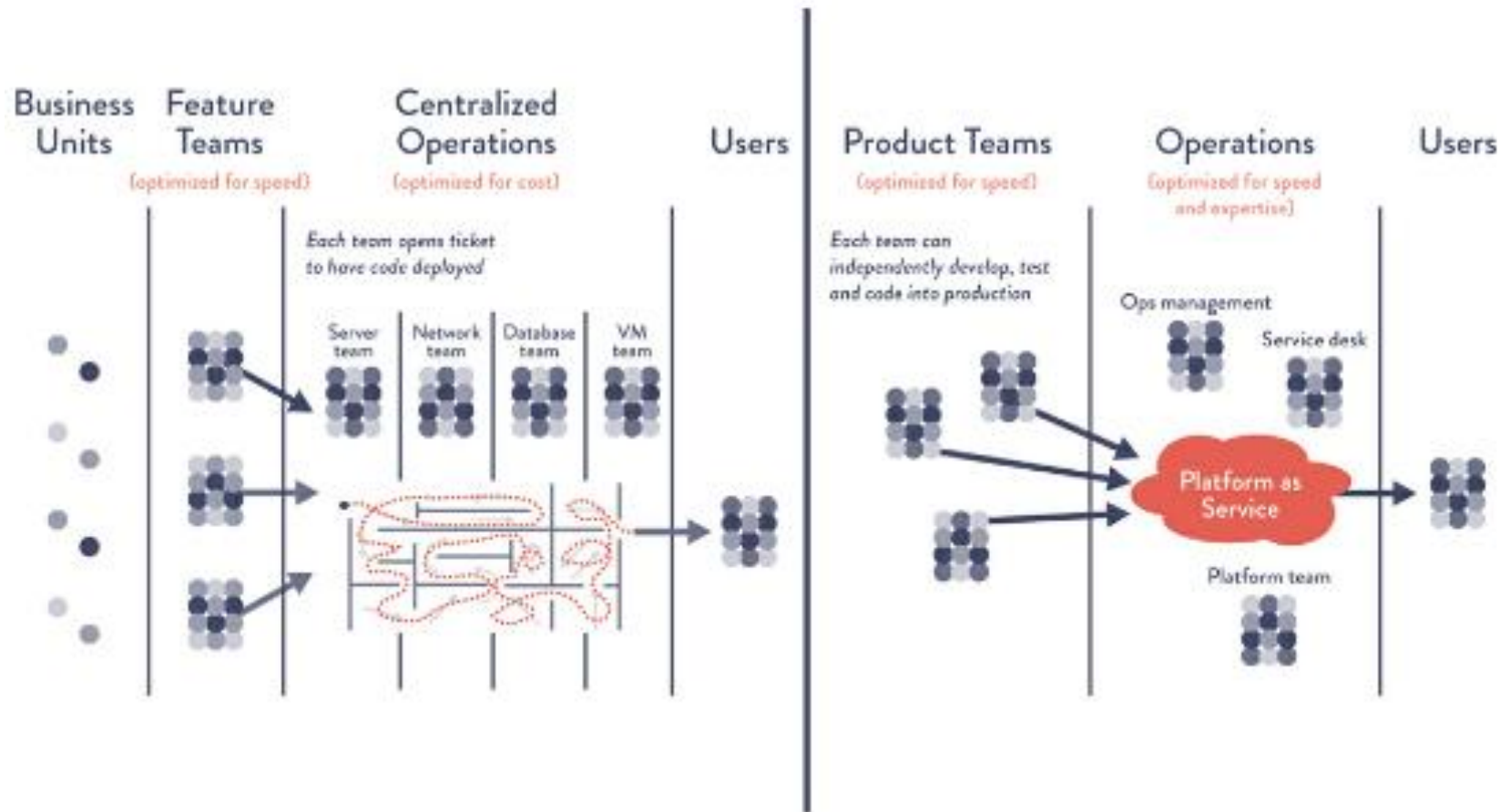
Market-oriented organizations optimize for responding quickly to customer needs. Tend to be flat, composed of multiple cross-functional disciplines (e.g., marketing, engineering, etc.), which often leads to potential redundancies across the organization

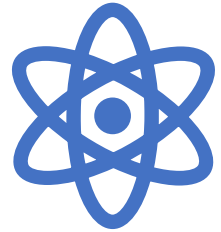
How to achieve market orientation

embed the functional engineers and skills (e.g., Ops, QA, Infosec) into each service team

create a platform organization that provides an automated technology platform for service teams to self-serve everything they need to test, deploy, monitor, and manage their services in testing and production environments

Making Functional Orientation Work





Testing, Operations,
and Security as
Everyone's Job Every
Day

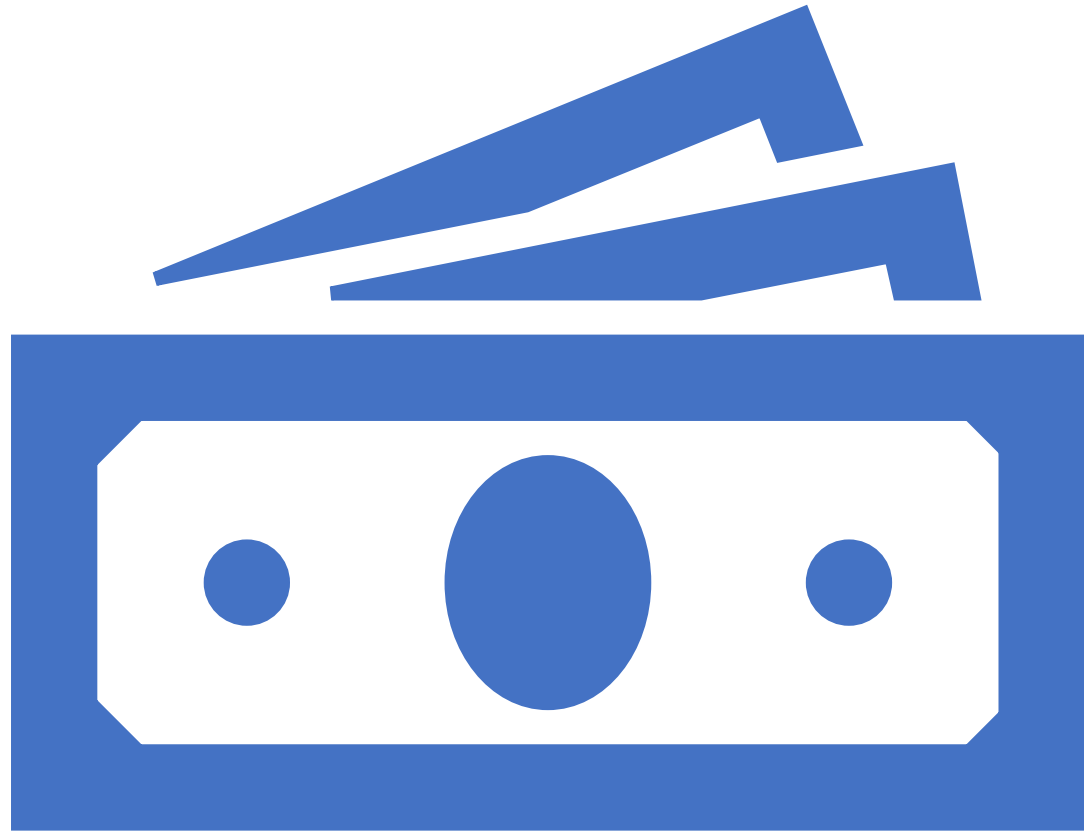


Enable Every
Team
Member to Be
a Generalist

"I-shaped" (Specialists)	"T-shaped" (Generalists)	"E-shaped"
Deep expertise in one area	Deep expertise in one area	Deep expertise in a few areas
Very few skills or experience in other areas	Broad skills across many areas	Experience across many areas Proven execution skills Always innovating
Creates bottlenecks quickly	Can step up to remove bottlenecks	Almost limitless potential
Insensitive to downstream waste and impact	Sensitive to downstream waste and impact	—
Prevents planning flexibility or absorption of variability	Helps make planning flexible and absorbs variability	—

Source: Scott Prugh, "Continuous Delivery," ScaledAgileFramework.com, February 14, 2013, <http://scaledagileframework.com/continuous-delivery/>.

Fund Not
Projects but
Services and
Products





Design Team Boundaries in Accordance with Conway's Law

software architecture should enable small teams to be independently productive, sufficiently decoupled from each other so that work can be done without excessive or unnecessary communication and coordination



Create Loosely Coupled Architectures to Enable Developer Productivity and Safety



Service Oriented Architecture



Services must be decoupled from other services and, just as important, from shared databases



Bounded context (book Domain-Driven Design by Eric J. Evans)

Keep Team Sizes Small (the “Two-Pizza Team” Rule)



reduce the amount of inter-team communication



encourage to keep the scope of each team's domain small and bounded

INTEGRATING OPERATIONS INTO THE DAILY WORK OF DEVELOPMENT

1

Create self-service capabilities to enable developers in the service teams to be productive

2

Embed Ops engineers into the service teams

3

Assign Ops liaisons to the service teams when embedding Ops is not possible

Create Shared Services to Increase Developer Productivity



Create a set of centralized platforms and tooling services that any Dev team can use to become more productive, such as getting production-like environments, deployment pipelines, automated testing tools, production telemetry dashboards, and so forth



Ensure that the platforms and services we create are the easiest and most appealing choices available



Build into these platforms the cumulative and collective experience of everyone in the organization, including QA, Operations, and Infosec



Look for internal toolchains that are widely being adopted in the organization

Embed Ops Engineers into Our Service Teams



reduce the reliance on centralized Operations



ops engineers become more closely connected to their internal and external customers



after the product is released to production, embedded Ops engineers may help with the production responsibilities of the Dev team



cross-train operations knowledge and expertise into a service team



transform operations knowledge into automated code that can be far more reliable and widely reused

Assign an Ops Liaison to Each Service Team

what the new product functionality is and why we're building it

how it works as it pertains to operability, scalability, and observability

how to monitor and collect metrics to ensure the progress, success, or failure of the functionality

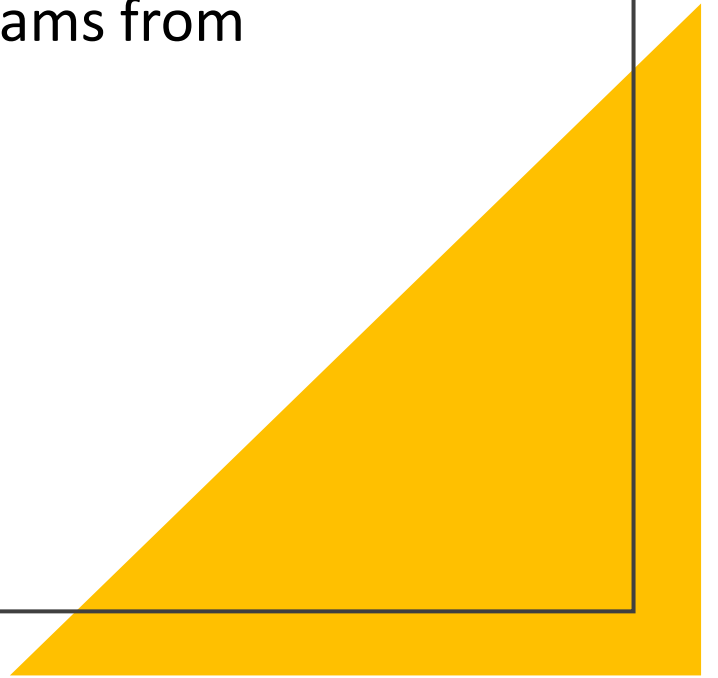
any departures from previous architectures and patterns, and the justifications for them

any extra needs for infrastructure and how usage will affect infrastructure capacity

feature launch plans

Risk

Ops liaisons are stretched too thin,
preventing the product teams from
achieving their goals



Integrate Ops into Dev Rituals



Invite Ops to Dev Standups (solve our current team problems before they become crisis)



Invite Ops to Our Dev Retrospectives (improve daily work is more important than daily work)



Make Relevant Ops Work Visible on Shared Kanban Boards (properly recognizing and integrating Ops work into all the relevant value streams)



Thank you